

IoT mikroservisi zasnovani na događajima

Uporedna evaluacija MQTT-a i Apache Kafke za edge/cloud scenarije

Arhitektura: Event-driven, 3 mikroservisa **Broker-i:** MQTT (Mosquitto) · Kafka (KRaft) **Model:** sensor_measurements

1 Kratak opis urađenog

Implementiran je asinhroni, event-driven sistem od tri mikroservisa (Ingestion, Storage, Analytics) koji istu logiku pokreće nad dva broker-a — **MQTT (Mosquitto)** i **Apache Kafka (KRaft režim, bez Zookeeper-a)**. Broker se bira profilom Docker Compose-a, a servisi koriste zajednički kod sa adapterima za oba broker-a (`BROKER` env).

- Ingestion (.NET):** simulira IoT uređaje; publikuje JSON merenja sa `produced_at` (epoch ms) i `device_id` (Kafka particioni ključ). Otporan na prekid mreže (reconnect u pozadini).
- Storage (.NET, Npgsql):** pretplata + grupni upis (binarni `COPY`) na svakih 500 poruka; opcionalno isključenje upisa (`STORAGE_WRITE_ENABLED=false`) za stres-testove.
- Analytics (Node.js/TS):** tumbling window od 10 s; prosek/maks temperature; **kritičan alarm** kada prosek pređe 50 °C; merenje end-to-end latencije.

Model podataka je isti kao u Projektu 1 (`sensor_measurements`), prošireno sa `device_id` , `recorded_at` (= `produced_at`) i `received_at` .

```
[Ingestion (.NET)] --publish--> [ BROKER ] --subscribe--> [Storage (.NET)] --batch INSERT (500)--> [PostgreSQL]
simulira uređaje      MQTT/Kafka topic      \--subscribe--> [Analytics (Node/TS)] --10s prozor--> ALERT
```

2 Konfiguracija eksperimenata

- MQTT:** QoS 0 / 1 / 2 (at most / at least / exactly once); perzistentne (trajne) pretplate.
- Kafka:** acks 0 / 1 / all; topik sa 3 particije; odvojene consumer grupe za Storage i Analytics.
- Alati:** emqtt-bench (`emqx/emqtt-bench`) za MQTT, `kafka-producer-perf-test.sh` za Kafku, `docker stats` za resurse.
- Okruženje:** Docker Desktop na macOS, ~7,6 GiB RAM dodeljeno VM-u. Scenariji A i C rađeni sa `STORAGE_WRITE_ENABLED=false` (da disk I/O ne bude usko grlo umesto brokera).

Napomena o uporedivosti: MQTT i Kafka brojevi nisu 1:1 (različiti alati). emqtt-bench meri publish-rate klijenata pod datim QoS-om; kafka-perf meri producer-rate pri `throughput=-1` (maksimalno guranje, pa se producer red zasiti — otud visoke producer-latencije).

3 Uporedna tabela performansi

Scenario A — Massive Sensor Ingestion (max throughput)

MQTT (emqtt-bench, 1 poruka / 10 ms po klijentu, payload 200 B)

QoS	100 uređaja	1000 uređaja	10000 uređaja
0 (at most once)	~18.700 msg/s	~186.000 msg/s	~235.000 msg/s
1 (at least once)	~10.000 msg/s	~60.000 msg/s	~54.000 msg/s
2 (exactly once)	~10.000 msg/s	~26.000 msg/s	—

Kafka (kafka-producer-perf-test, 300.000 poruka, payload 200 B, throughput=-1)

acks	Throughput	p95 latencija (producer→broker)
0	~202.700 msg/s (38,7 MB/s)	495 ms
1	~233.600 msg/s (44,6 MB/s)	360 ms
all	~205.200 msg/s (39,1 MB/s)	552 ms

Zapažanja A:

- MQTT pokazuje čist trade-off: pri 1000 klijenata QoS 0 (~186k) >> QoS 1 (~60k) >> QoS 2 (~26k). Pri samo 100 klijenata (10k msg/s cilj) broker nije zasićen pa QoS 1 i 2 lako drže 10k/s.
- QoS 0 skalira sa brojem klijenata (18,7k → 186k → 235k); QoS 1 pravi plato ~55-60k (ack-bound).
- Kafka je ravnomernija po acks nivoima jer na **jednom** KRaft brokeru nema mrežne replikacije (acks=all ≈ acks=1). Apsolutni throughput je visok jer Kafka batch-uje i sekvencijalno piše u log.

Zbirna tabela (Throughput / p95 transit-latencija / Broker CPU / Broker RAM)

p95 je end-to-end **transit** latencija (od `produced_at` do prijema u Analytics) pri ~1-10k msg/s.

Broker (config)	Throughput	p95 transit	Broker CPU	Broker RAM
MQTT QoS 0	~186.000 msg/s	~1 ms	~94 % (1 jezgro)	~ 39 MiB
MQTT QoS 1	~60.000 msg/s	~1 ms	~94 %	~39 MiB
MQTT QoS 2	~26.000 msg/s	~1 ms	~94 %	~39 MiB
Kafka acks 0	~202.700 msg/s	~21 ms	~23 % @1k/s	~ 326 MiB
Kafka acks 1	~233.600 msg/s	~21 ms	~23 %	~326 MiB
Kafka acks all	~205.200 msg/s	~21 ms	~23 %	~326 MiB

Footprint servisa: Storage/Ingestion (.NET) ~24–26 MiB; Analytics (Node) ~49–130 MiB.
Ključno: Mosquitto ~15 MiB (idle) - 39 MiB (10k klijenata); Kafka (JVM) ~326 MiB → **~8-20x više memorije**.

Scenario B — Edge Connectivity Failures (prekid mreže 30 s na simulatoru)

MQTT		Kafka
Throughput tokom prekida	pad ~994 → ~87 msg/s	producer baferuje (librdkafka)
Oporavak	„veza obnovljena“, ~1024 msg/s	consumer lag → 0 (catch-up po offset-u)
Gubitak poruka	DA — poruke generisane dok je producent offline su izgubljene (QoS ne pomaže)	NE — baferovane poruke isporučene po povratku

Scenario C — Burst Event Load (50 → 5000 msg/s)

MQTT		Kafka
Burst	5000/s apsorbovan trenutno (storage pik ~6000/s)	5000/s održan
Backlog	nema (kapacitet ~60k/s ≫ 5k)	consumer lag ograničen (sawtooth ~0-1600)
Recovery time	trenutan	kontinuiran, lag se vraća ka 0 svaki ciklus

Scenario D — Real-Time Alerting (end-to-end latencija)

Broker	Transit p50 / p95		Latencija alarma (e2e)
MQTT	0 ms / 1 ms		~9.970 ms
Kafka	13 ms / 21 ms		~10.330 ms

Latencija alarma (~10 s) dominira veličinom tumbling prozora (alarm čeka zatvaranje prozora) i slična je za oba broker-a. Pravi pokazatelj brzine brokera je **transit** latencija: MQTT ~1 ms ≪ Kafka ~21 ms (Kafka troši na consumer poll/batching) → MQTT je pogodniji za real-time alerting.

4 Odgovori na inženjerska pitanja

1. Zašto je MQTT idealan na edge uređajima, a neadekvatan za istorijsku analitiku velikih podataka?

MQTT je lagan pub/sub protokol nad TCP-om sa minimalnim zaglavljem, **vrlo malim memorijskim otiskom** (izmereno: Mosquitto ~15–39 MiB čak i pri 10.000 klijenata) i najnižom latencijom (transit p95 ~1 ms). Idealan je za senzore i edge gateway-e gde su resursi i veza ograničeni. Postaje neadekvatan za istorijsku analitiku jer je broker **prolazan** — ne čuva trajni, ponovljivo-čitljiv log sa offset-ima i particijama; poruke koje nisu odmah potrošene se gube (što je Scenario B i potvrdio: poruke poslate dok je veza pala su izgubljene). Nema replay, horizontalno skaliranje potrošača ni dugotrajno skladištenje toka.

2. Zašto Kafka dominira u data-intensive cloud sistemima i kolika je „cena“ skalabilnosti?

Kafka je distribuirani, partitionisani **commit log**: poruke se trajno čuvaju, mogu se ponovo čitati (replay), particije daju horizontalnu skalabilnost, a consumer grupe + offset-i pouzdanu obradu „at least once“ (Scenario B: lag se vratio na 0 bez gubitka, librdkafka baferovao tokom prekida). „Cena“ je u resursima: izmereni RAM otisak brokera **~326 MiB (JVM)** naspram Mosquitto ~15–39 MiB — **~8-20x više memorije** za isti tok. Na hardverski ograničenim edge serverima to je teško opravdati; Kafka pripada cloud-u, gde su trajnost i propusnost važniji od otiska.

3. Uporedna tabela performansi

Vidi sekciju 3 — sve vrednosti su izmerene.

5 Zaključak

- MQTT**: minimalni otisak (~15–39 MiB), najniža latencija (~1 ms), čist QoS trade-off (186k → 60k → 26k msg/s). Ali bez trajnosti — poruke se gube pri prekidu na strani producenta. Najbolji za **edge**: senzori, real-time alerting, ograničeni resursi.

- **Kafka:** visok i ravnomeran throughput (200–234k msg/s), trajnost i replay, oporavak bez gubitka preko offset-a — po ceni ~8–20× veće memorije i nešto više latencije (~21 ms). Najbolja za **cloud**: istorijska analitika, više nezavisnih potrošača, velika propusnost.
- **Ključni trade-off** (latencija vs pouzdanost): viši MQTT QoS i Kafka acks $\geq$ 1 daju jače garancije isporuke po ceni manjeg throughput-a; QoS 0 / acks 0 su najbrži ali bez garancija.